UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

APUNTES

# Sistemas Distribuidos

## Clase 03 – Consistencia y replicación

**Departamento de Computación**

Ingeniería Informática – Comisión A

Pilar – Primer cuatrimestre 2026

**Equipo docente**

11 de agosto de 2026

# 1 Introducción

Una vez resuelta la comunicación (RPC), el problema central es el **estado**. Para mejorar disponibilidad y latencia, los sistemas replican datos. Esto introduce una tensión fundamental:

> No es posible mantener múltiples copias perfectamente sincronizadas sin coordinación.

# 2 Modelo

Consideremos un objeto replicado en $N$ nodos. Las operaciones son:

- `write(k, v)`
- `read(k)`

La red es asíncrona: mensajes pueden retrasarse, perderse o reordenarse.

# 3 Divergencia

Dos réplicas $R_i$ y $R_j$ divergen si almacenan valores distintos para la misma clave en el mismo tiempo lógico. Esto ocurre por:

- latencias distintas
- concurrencia
- particiones

# 4 Consistencia

## 4.1 Linearizabilidad

Una historia es linearizable si existe un orden total que:

- respeta el tiempo real
- es consistente con la especificación secuencial

**Propiedad** Cada operación tiene un punto de linearización entre invocación y respuesta.

## 4.2 Consistencia causal

Definimos la relación *happens-before* ($\rightarrow$):

- orden de programa
- envío/recepción de mensajes
- transitividad entre relaciones conocidas

La consistencia causal exige que si $a \rightarrow b$, entonces todo proceso observa $a$ antes que $b$.

# 5 Tiempo físico, orden lógico y causalidad

En sistemas distribuidos no existe un reloj global perfecto que permita ordenar todos los eventos con certeza. Cada nodo observa su propio reloj local, la red introduce demoras variables y los mensajes pueden llegar tarde, perderse o reordenarse. Por eso, muchas decisiones de diseño deben basarse en causalidad lógica y no solamente en timestamps físicos.

> La pregunta importante no siempre es "qué hora marcaba el reloj", sino "qué evento pudo influir sobre qué otro evento".

Comparar timestamps físicos entre nodos distintos puede ser engañoso:

- los relojes locales pueden derivar;
- una corrección de sincronización puede mover un reloj hacia adelante o hacia atrás;
- un mensaje enviado antes puede llegar después por latencia variable;
- dos eventos pueden tener timestamps comparables sin que exista relación causal entre ellos.

## 5.1 Relojes de Lamport

Los relojes de Lamport asignan un contador lógico a cada evento. Cada proceso mantiene un contador $L_i$:

- antes de un evento local, incrementa $L_i$;
- al enviar un mensaje, adjunta el valor actual;
- al recibir un mensaje con timestamp $t$, actualiza $L_i \leftarrow \max(L_i, t) + 1$.

La propiedad central es:

$$a \to b \implies L(a) < L(b)$$

La implicación inversa no vale: si $L(a) < L(b)$, no necesariamente $a \to b$. Lamport preserva causalidad conocida, pero no detecta concurrencia con precisión.

## 5.2 Consistencia eventual

Garantiza convergencia:

> Si no hay nuevas escrituras, todas las réplicas convergen al mismo valor.

No garantiza orden ni lecturas recientes.

# 6 Arquitecturas de replicación

## 6.1 Leader-based

Un líder serializa escrituras. Replicación:

- síncrona: mayor consistencia, más latencia
- asíncrona: menor latencia, riesgo de pérdida

### 6.2 Multi-leader

Cada región acepta escrituras. Problema:

- conflictos concurrentes

### 6.3 Leaderless (Dynamo)

Parámetros:

- $N$: número de réplicas
- $W$: write quorum
- $R$: read quorum

**Intersección** Si $R + W > N$, los quorums se intersectan.

## 7 Conflictos

Dos escrituras concurrentes generan versiones divergentes.

### 7.1 Versionado

Vector clocks:

- detectan concurrencia
- permiten merge explícito

Cada proceso mantiene un vector con una componente por proceso. Para comparar dos vectores $V(a)$ y $V(b)$:

- $V(a) < V(b)$ si todas las componentes de $V(a)$ son menores o iguales que las de $V(b)$ y al menos una es estrictamente menor;
- si $V(a) < V(b)$, entonces $a$ precede causalmente a $b$;
- si los vectores no son comparables, los eventos son concurrentes.

Por ejemplo, $(2, 1, 0)$ y $(1, 2, 0)$ no son comparables; representan escrituras concurrentes. En un store replicado, esto indica que no conviene sobrescribir ciegamente una versión con otra, sino conservar ambas o aplicar reconciliación semántica.

### 7.2 Resolución

- LWW: simple, puede perder datos
- merge semántico

## 8 Anti-entropy

Mecanismos:

- gossip
- read-repair

- hinted handoff

## 9 Trade-offs

- consistencia fuerte → coordinación → latencia
- consistencia débil → disponibilidad → anomalías

## 10 Marco $D = (P, F, G, S, T)$

- $P$: mantener estado replicado
- $F$: delay, pérdida, partición
- $G$: modelo de consistencia
- $S$: arquitectura (leader, quorum)
- $T$: latencia, disponibilidad, complejidad

## 11 Conclusión

La consistencia no es una propiedad binaria: es un contrato explícito entre el sistema y sus clientes.

## 12 Síntesis razonada de la clase

Esta unidad debe leerse como parte de una cadena de decisiones de diseño y no como un catálogo de mecanismos. Para estudiar el tema con profundidad conviene reconstruir cuatro preguntas: *qué problema aparece*, *qué garantía se desea*, *qué mecanismo la aproxima* y *qué costo introduce*. En cada ejemplo debe identificarse además el modelo de falla implícito.

Una respuesta técnicamente madura no termina al nombrar un algoritmo o patrón: declara sus supuestos, la propiedad que preserva y el escenario en el que deja de ser suficiente.

## 13 Bibliografía guiada y ruta de profundización

Para profundizar esta clase se recomienda: Kleppmann, caps. 5, 8 y 9; Lamport sobre tiempo y orden.

La lectura debe hacerse con preguntas concretas: (1) ¿qué modelo de sistema asume el autor?, (2) ¿qué propiedad demuestra o persigue?, (3) ¿qué falla o anomalía motiva el mecanismo?, y (4) ¿qué trade-off queda abierto?

## 14 Conexión con el Trabajo Final Integrador

El grupo debe señalar qué parte de su sistema utiliza los conceptos de esta unidad, justificar por qué son necesarios y documentar una alternativa descartada. Cuando el contenido todavía no corresponda a la etapa vigente, debe registrarse como hipótesis o decisión pendiente.

# 15 Autoevaluación conceptual

1. ¿Cuál es el problema distribuido concreto que motiva esta clase?
2. ¿Qué propiedad de seguridad y qué propiedad de progreso están en juego?
3. ¿Qué supuesto de red, tiempo o falla resulta decisivo?
4. ¿Qué trade-off aceptaría en un sistema real y cuál no aceptaría en un dominio crítico?
5. ¿Cómo observaría experimentalmente que la solución se comporta como espera?

# 16 Profundización autocontenida v9

## 16.1 Problema, límite e invariante

El núcleo de esta clase es replicación síncrona y asíncrona, stale reads, quórums, causalidad y modelos de consistencia. Para estudiarlo de manera autónoma conviene separar tres planos. Primero, el *problema observable*: qué comportamiento incorrecto puede ver un cliente o un operador. Segundo, el *límite del modelo*: qué información, sincronía o coordinación no está disponible. Tercero, el *invariante*: qué propiedad no debe violarse aunque el sistema pierda progreso temporalmente.

El modelo de fallas relevante incluye réplica atrasada, escritura concurrente, partición y failover. Una solución debe describirse como protocolo y no solo como nombre de tecnología: actores, estados, mensajes, condición de éxito, condición de aborto y estado visible después de una interrupción. La evidencia esperada es historia de lecturas/escrituras, staleness y comparación de quórums.

## 16.2 Método de análisis

Ante un caso nuevo, siga esta secuencia:

1. Identifique actores, dato o recurso compartido y frontera de confianza.
2. Escriba una ejecución nominal y una ejecución adversarial.
3. Distinga seguridad (lo malo no ocurre) de progreso (algo bueno finalmente ocurre).
4. Declare qué supuesto permite avanzar: mayoría, deadline, sincronía parcial, operación idempotente, almacenamiento durable u otro.
5. Compare una alternativa más simple y una más fuerte; registre qué métrica o experimento haría cambiar la decisión.

## 16.3 Caso transferible

Considere una plataforma de reservas que recibe operaciones duplicadas, pierde conectividad entre regiones y debe sostener un camino crítico sin ocultar el estado real al usuario. Aplique el mecanismo de esta clase a una única decisión concreta. Una respuesta madura no intenta resolver todo: delimita la operación, formula la garantía y explica la degradación esperada cuando el supuesto central deja de cumplirse.

## 16.4 Errores de razonamiento frecuentes

- confundir una propiedad con el producto que suele implementarla;
- describir el camino feliz sin recorrer una falla intermedia;

- afirmar “exactly-once”, “alta disponibilidad” o “consistencia” sin definir alcance observable;
- medir solo throughput aunque la hipótesis trate sobre semántica o recuperación;
- presentar la complejidad añadida como beneficio sin compararla con la alternativa más simple.

## 17 Lectura guiada

Kleppmann, capítulo 5; lecturas sobre consistencia causal y quórums.

Durante la lectura, registre: modelo de sistema, propiedad perseguida, contraejemplo que motiva el mecanismo, costo principal y una condición bajo la cual no lo elegiría. Estas cinco anotaciones convierten la bibliografía en una herramienta de diseño y no en una lista para memorizar.

## 18 Aplicación al Trabajo Final Integrador

El equipo debe producir un ADR breve con la decisión asociada a esta clase. Debe contener: contexto, dos alternativas, supuesto decisivo, garantía elegida, trade-off, evidencia pendiente y fecha de revisión. La evidencia mínima esperada para esta unidad es: historia de lecturas/escrituras, staleness y comparación de quórums.

## 19 Autoevaluación avanzada

1. ¿Puedo explicar la clase sin nombrar primero una tecnología?
2. ¿Puedo construir una ejecución que viole la garantía si el mecanismo se configura mal?
3. ¿Puedo distinguir seguridad, progreso y experiencia del usuario?
4. ¿Puedo proponer una medición que valide la propiedad, no solo el rendimiento?
5. ¿Puedo defender cuándo una solución más simple sería suficiente?